Task 3: Design Your First App Idea (Concept + UI Sketch)

Name: Sachin

Course Title: [Android App Development Using GenAi]

1. **App Concept: "EcoTrack: Personal Sustainability Assistant"**

Core Problem: Many people want to live more sustainably but find it difficult to track their daily carbon footprint or find eco-friendly alternatives for common products.

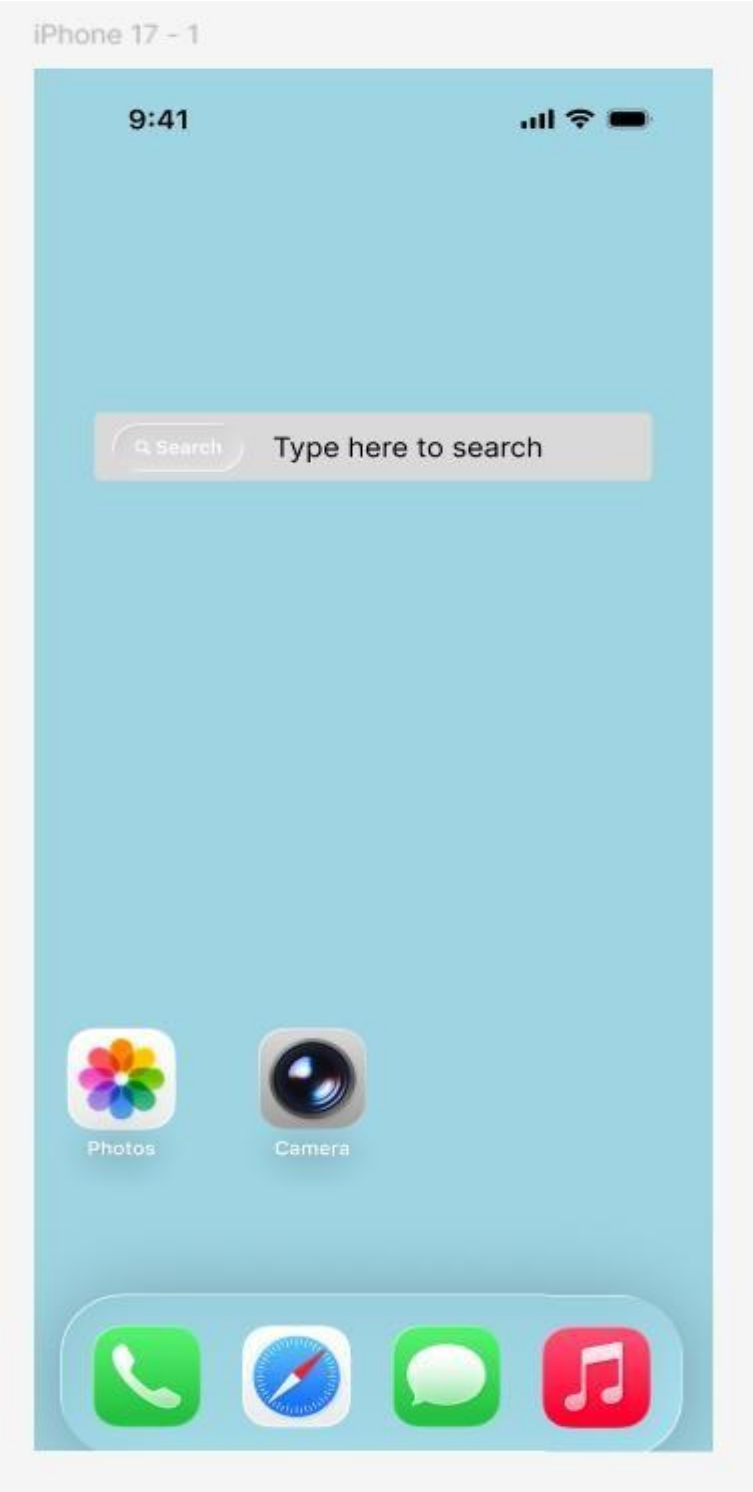
Target Audience: Environmentally conscious students and professionals (like yourself) who use Android devices to manage their daily routines.

Key Features:

- **Smart Scanner:** Uses the phone's camera to scan product barcodes and provide an "Eco-Score" based on packaging and sourcing.
- **Daily Log:** A simple checklist for sustainable actions (e.g., used public transport, recycled, avoided single-use plastic).
- **Community Maps:** Integration with Google Maps to show the nearest recycling centers or zero-waste stores.

UI Element	Android Component	Purpose
Top Bar	Toolbar / AppBarLayout	Displays the app name and a settings icon.
Daily Progress	ProgressBar (Circular)	Shows a visual percentage of the user's daily eco-goal met.
Feature Buttons	FloatingActionButton	A prominent "+" button to quickly log a new eco-action.
Action List	RecyclerView	A scrollable list showing the user’s recent sustainable activities.
Navigation	BottomNavigationView	Icons to switch between 'Home', 'Scan', 'Map', and 'Profile'.

2. UI Sketch / Wireframe Description



3. Design Reflection:

In 5–6 lines, answer:

1. Why did you choose this app idea?

   I chose this app because I have previously worked with Figma during my academics. Since I am already familiar with the tool, I feel it is a suitable platform to design and develop a simple mobile app home screen. Therefore, I selected this app.

2. How does your design make it simple or enjoyable to use?

   The design process is simple and engaging. This app allows me to search for the required elements for the prototype and independently develop the design based on my own ideas.

3. Which Compose UI elements (like Text, Column, Button, Image) would you likely use later to build this UI?

   I would like to use the text.

```
code : // ----------------- //
Student Solution
// -----------------

// 1) Data class
data class Profile(val name: String, val title: String, val bio: String)

// 2) Render Avatar (ASCII placeholder) fun
renderAvatar(width: Int, height: Int): String {
val builder = StringBuilder()
   val contentWidth = width - 2

   builder.append("|" + " ".repeat(contentWidth) + "|\n")

   for (i in 1 until height - 1) {
      val text = if (i == height / 2) "Avatar ${width}x$height" else ""
val trimmed = text.take(contentWidth)
      val leftPadding = (contentWidth - trimmed.length) / 2        val
rightPadding = contentWidth - trimmed.length - leftPadding

      builder.append("|" +
```

```
            " ".repeat(leftPadding) +
trimmed +
            " ".repeat(rightPadding) +
            "|\n")
   }

   builder.append("|" + " ".repeat(contentWidth) + "|\n")

   return builder.toString()
}

// 3) Render Text Block
fun renderTextBlock(label: String, text: String, width: Int): String {
val contentWidth = width - 4   // for "| " and " |"     val fullText =
"$label: $text"     val trimmed = fullText.take(contentWidth)     val
padding = contentWidth - trimmed.length

   return "| $trimmed${" ".repeat(padding)} |\n"
}

// 4) Render Profile Card fun
renderProfile(     profile:
Profile,     width: Int = 30,
showAvatar: Boolean = true
): String {
   val builder = StringBuilder()

   val border = "+" + "-".repeat(width - 2) + "+\n"
builder.append(border)

   if (showAvatar) {
      builder.append(renderAvatar(width, 5))
   }

   builder.append(renderTextBlock("Name", profile.name, width))
builder.append(renderTextBlock("Title", profile.title, width))
builder.append(renderTextBlock("Bio", profile.bio, width))

   builder.append(border)

   return builder.toString()
}

// 5) Main Function fun
main() {

   val profile1 = Profile(
"Ada Lovelace",
      "Software Dev",
```

```
        "Early computing pioneer"
    )

    val profile2 = Profile(
"Alan Turing",
        "Mathematician",
        "Father of theoretical CS"
    )

    val profile3 = Profile(
        "Katherine Johnson",
        "NASA Scientist",
        "Orbital mechanics expert"
    )

    println(renderProfile(profile1, 30, true))
  println(renderProfile(profile2, 30, false))
  println(renderProfile(profile3, 30, true))
```

}

Task 1 – Text Composables Understanding & Exploration

1. What is a @Composable Function?

A `@Composable` function is a special Kotlin function used in Jetpack Compose to define UI elements. It describes how a part of the screen should look and behave. Instead of using XML layouts, Compose builds UI directly using these composable functions in Kotlin.

---

2. Three Key Text Composables

1. Text

What it does:
Displays text on the screen. Use-case:

Example
Showing a title like “Welcome” or displaying a paragraph description.

---

2. Column

What it does:
Arranges UI elements vertically (top to bottom). Use-case:

Example
Placing Title → Subtitle → Description one below another.

3. Row

What it does:
Arranges UI elements horizontally (side-by-side).

Example Use-case:
Placing two buttons next to each other.

3. How Column and Row Affect Text Appearance

Column stacks text elements vertically.
Row places elements side-by-side horizontally.

This directly affects how content is structured and read on the screen.

4. Wireframe Description

Screen Layout Plan:

Column
→ Text (Title – 24sp, Bold)
→ Text (Subtitle – 18sp)
→ Text (Paragraph – 16sp)
→ Row
→ Button (Confirm)
→ Button (Cancel)

Spacing:

- 16dp between title and subtitle
- 8dp between subtitle and paragraph
- 12dp between buttons

5. Reflection

Planning the layout visually before coding helps avoid design mistakes. It improves clarity and structure. It reduces trial-and-error during implementation. Visual planning ensures proper spacing, alignment, and readability before writing code.

Task 2 – Image Inclusion & Resource Management in Compose

1. What is Image Composable?

The `Image()` composable is used in Jetpack Compose to display images from drawable resources or other sources. It allows customization such as size, scaling, and content description.

Example use-case: Displaying a profile picture at the top of a screen.

---

2. Why Image Density Matters

Android devices have different screen densities. If density is ignored:

- Images may appear blurry.
- UI layout may break.
- Performance may reduce due to large file sizes.

Using proper folders like `drawable-nodpi` prevents unwanted scaling.

---

3. Best Practices

1. Use meaningful file names (for example, profile_image.png).
2. Optimize image size to reduce memory usage.
3. Add contentDescription for accessibility.
4. Avoid using very high-resolution raw images unnecessarily.

---

4. Storyboard Description

Frame 1 – Text Only

Column → Title → Subtitle → Description

Frame 2 – Image Added

Column → Image (Top) → Text Content Adjusted

Frame 3 –
Column → Full-width Image → Text Content Image

This shows how image placement changes layout balanc

and readability.

5. Reflection

Images improve UI design but can affect performance if not optimized. Large images increase loading time. Missing contentDescription reduces accessibility. Proper planning ensures efficient and clean UI design.

Task 3 – Planning a Simple Layout with Text + Image

App Concept: User Profile Screen

Purpose

To display user profile details including image, name, bio, and action buttons.

Layout Structure

Column
→ Image (Profile Picture – Top, 120dp)
→ Text (Name – 24sp, Bold)
→ Text (Bio – 16sp)
→ Row
→ Button (Follow)
→ Button (Message)

Spacing:

- 16dp between image and title
- 8dp between text blocks
- 12dp between buttons

Resource Plan

Image:

- File Name: profile_pic.png

- Folder: drawable-nodpi
- Size: Medium (optimized)

Text:

- Title: 24sp, Bold
- Subtitle: 18sp
- Body: 16sp
- Color: primaryTextColor

Reflection

Selecting appropriate composables and resources before coding saves time and reduces layout errors. Planning spacing and structure improves clarity. Proper preparation ensures smoother implementation and better UI quality.

Task 4 – Accessibility, Performance & Review

Review Table

Element	Potential Issue	Proposed Fix	Priority
Profile Image	Large file size	Compress and resize	High
Subtitle	Small font size	Increase to 18sp	Medium
Buttons	Small touch area	Minimum 48dp height	High
Image	Missing contentDescription	Add accessibility text	High

Mini Action Plan

Before Coding:

- Optimize image file size.
- Increase subtitle font size.
- Add contentDescription.

During Coding:

- Maintain consistent padding (16dp standard).
- Use proper scaling such as ContentScale.Crop.

After Testing:

- Check layout on small and large screens.
- Improve contrast if required.

Reflection

Planning accessibility and performance early improves overall app quality. It ensures inclusivity and better usability. Early review reduces redesign effort later and results in efficient and well-structured Compose applications